DB NORM – An Efficient Database Normalizer

A THESIS SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR
THE DEGREE OF

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Name	Univ. Roll No.
Subham Kumar Shaw	10800122102
Sanjeevni Srivastava	10800122103
Vivek Mahato	10800122098
Apurba Pal	10800122039

UNDER THE GUIDANCE OF

Dr. MONISH CHATTERJEE
(Professor & HoD, Dept of Computer Science and Engineering)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ASANSOL ENGINEERING COLLEGE
AFFILIATED TO
MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY

July, 2026

Contents

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASANSOL ENGINEERING COLLEGE

Vivekananda Sarani, Kanyapur, Asansol, West Bengal – 713305

Certificate of Recommendation

I hereby recommend that the thesis entitled, **"DB NORM – An Efficient Database Normalizer"** carried out under my supervision by the group of students listed below may be accepted in partial fulfilment of the requirement for the degree of "Bachelor of Technology in Computer Science and Engineering" of Asansol Engineering College under MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY.

Name	Univ. Roll No.
SUBHAM KUMAR SHAW	10800122102
SANJEEVNI SRIVASTAVA	10800122103
VIVEK MAHATO	10800122098
APURBA PAL	10800122039

.....

Dr. Monish Chatterjee
Thesis Supervisor
Dept. of Computer Science and Engineering,
Asansol Engineering College,
Asansol-713305

Countersigned:

.....

Dr. Monish Chatterjee
Head of the Department
Dept. of Comp. Sc. & Engg,
Asansol Engineering College,
Asansol-713305

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ASANSOL ENGINEERING COLLEGE
Vivekananda Sarani, Kanyapur, Asansol, West Bengal – 713305

Certificate of Approval

The thesis is hereby approved as creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance in the partial fulfilment of the degree for which it has been submitted. It is understood that by this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein but approve the thesis only for the purpose for which it is submitted.

M. Chatterjee

.....

Dr. Monish Chatterjee
Thesis Supervisor

Dept. of Computer Science and Engineering,
Asansol Engineering College,
Asansol-713305

Acknowledgement

It is our great privilege to express our profound and sincere gratitude to our Thesis Supervisor, **Dr. Monish Chatterjee** for providing us very cooperative and valuable guidance at every stage of the project work being carried out under his supervision. His valuable advice and instructions in carrying out the present study have been a very rewarding and pleasurable experience that has greatly benefited us throughout the period of work.

We would like to convey our sincere gratitude towards Dr. Monish Chatterjee, Head of the Department, Asansol Engineering College for providing us the requisite support for timely completion of our work. We would also like to pay our heartiest thanks and gratitude to all the teachers of the Department of Computer Science and Engineering, Asansol Engineering College for various suggestions being provided in attaining success in our work.

We would like to express our earnest thanks to Mr. Suman Mallick, of CSE Project Lab for his technical assistance provided during our project work.

Finally, I would like to express my deep sense of gratitude to my parents for their constant motivation and support throughout my work.

Subham Kr. Shaw
.....
SUBHAM KUMAR SHAW

Sanjeevni Srivastava
.....
SANJEEVNI SRIVASTAVA

Vivek Mahato.
.....
VIVEK MAHATO

Apurba Pal
.....
APURBA PAL

Abstract

Database normalization plays a crucial role in the design of efficient, reliable, and scalable relational database systems by eliminating redundancy, preventing update anomalies, and ensuring data integrity. Although the theoretical concepts of normalization are well established, students and practitioners often face challenges in applying normalization techniques systematically, particularly when computing attribute closures, deriving minimal covers, identifying candidate keys, analyzing functional and multivalued dependencies, and performing lossless and dependency-preserving decompositions across multiple normal forms.

To address these challenges, **DB Norm** is proposed as a highly interactive web-based educational and analytical tool that automates the complete normalization workflow. Developed using a modern full-stack architecture comprising a **React + Vite frontend** and a **Spring Boot backend**, the system integrates advanced algorithms for closure computation, minimal cover generation, candidate key discovery, dependency analysis, and automated schema decomposition. Beyond automation, DB Norm emphasizes conceptual understanding by providing step-by-step explanations, graphical visualizations, dependency diagrams, and detailed decomposition justifications.

The tool supports normalization from **First Normal Form (1NF)** through **Fifth Normal Form (5NF)**, including **Second Normal Form (2NF)**, **Third Normal Form (3NF)**, **Boyce–Codd Normal Form (BCNF)**, and **Fourth Normal Form (4NF)**. It handles functional dependencies, multivalued dependencies, and join dependencies, ensuring that decompositions are lossless, dependency-preserving, and theoretically sound.

This report presents the theoretical foundations of relational database normalization, discusses the algorithms implemented within DB Norm, and evaluates their correctness and efficiency using complex relational schemas. A comprehensive case study based on the schema **R = {A, B, C, D, E, F}** with functional dependencies **F = {AB→C, C→D, D→E, A→F}** is analyzed to demonstrate the complete normalization process. Experimental results indicate that DB Norm significantly improves both learning outcomes and normalization accuracy while reducing the time required for manual computations.

The proposed system serves as an effective educational platform for students, instructors, and database practitioners by combining algorithmic rigor with interactive learning. The report concludes with performance analysis, limitations, and future enhancements, including support for advanced dependency discovery, SQL schema integration, AI-assisted normalization guidance, and collaborative learning features.

List of Figures

1. PREFACE

This project has been undertaken as part of the final-year academic requirements for the Bachelor of Technology degree in Computer Science and Engineering. The report reflects a synthesis of theoretical rigor, algorithmic clarity, and modern software-engineering practices. It presents a comprehensive full-stack solution to a classical problem in database design: the systematic normalization of a relational schema based on explicitly provided functional dependencies.

The project aims not only to implement normalization automation but also to illustrate the complete reasoning process—making it a superior pedagogical tool compared to conventional normalization calculators.

1.1 Introduction

Database normalization is a key process in relational database design that minimizes redundancy, prevents anomalies, and improves data integrity. It involves transforming a database schema into well-structured relations that satisfy normal forms from 1NF to 5NF. However, students often face difficulties in computing closures, identifying candidate keys, deriving minimal covers, and performing correct decompositions.

DBNorm is a web-based educational tool that automates the normalization process up to 5NF. Built with React + Vite and Spring Boot, it provides functional dependency analysis, candidate key generation, step-by-step decomposition, graphical visualizations, and detailed explanations, making database normalization easier to learn and apply.

DB Norm addresses these concerns through:

- Automated FD analysis
- Detailed algorithmic explanations
- Stepwise decomposition
- Graphical visualizations
- High-quality UX for learning

1.2 Motivation of the Project

Database normalization is a fundamental concept in relational database design that ensures data consistency, reduces redundancy, and improves query efficiency. Despite its importance, learners often find normalization challenging due to:

- Abstract mathematical formulations
- Multi-step reasoning in identifying dependencies
- Complexity in real-world functional dependencies

Manual normalization exercises are time-consuming and error-prone, especially when functional dependencies involve:

- Composite Determinants (e.g., AB → C)
- Transitive Chains (e.g., C → D → E)
- Additional Dependencies (e.g., A → F)

These complexities introduce issues such as:

- Partial Dependencies – critical for Second Normal Form (2NF)
- Transitive Dependencies – need resolution for Third Normal Form (3NF)
- BCNF Violations – require decomposition for optimal design

Why This Project?

The project schema deliberately incorporates such dependencies to demonstrate real-world normalization challenges. DB Norm is designed as an interactive tool that bridges theory and practice. Key features include: Key Features:

- Partial Dependency Detection and Resolution – eliminates 2NF violations.
- Transitive Dependency Identification – simplifies conversion to 3NF.
- BCNF Violation Detection and Decomposition – handles complex functional dependencies.
- Multivalued Dependency (MVD) Analysis – supports decomposition into 4NF.
- Join Dependency Detection and Decomposition – enables normalization up to 5NF.
- Candidate Key and Minimal Cover Generation – automates critical normalization steps.
- Visual Dependency Diagrams and Stepwise Explanations

1.3 Basic Description of the Project

DB Norm – An Efficient Database Normalizer is a full-stack intelligent web application that automates the complete process of database normalization from 1NF to 5NF. The system is designed to reduce data redundancy, eliminate update anomalies, maintain data integrity, and improve overall database efficiency. By integrating advanced normalization algorithms with an interactive user interface, DB Norm simplifies one of the most challenging aspects of relational database design.

The system accepts a relational schema along with its functional dependencies and performs comprehensive dependency analysis to determine the highest normal form satisfied by the schema. It then automatically normalizes the relation through 2NF, 3NF, BCNF, 4NF, and 5NF, ensuring lossless decomposition and, whenever possible, dependency preservation. In addition to functional dependencies, the system supports the analysis of multivalued dependencies (MVDs) and join dependencies (JDs) required for higher normal forms.

DB Norm follows a structured computational pipeline consisting of attribute closure computation, minimal cover generation, candidate key identification, dependency analysis, and normalization synthesis. The system detects partial dependencies, transitive dependencies, BCNF violations, multivalued dependencies, and join dependencies, enabling users to understand the rationale behind each decomposition step.

The backend, developed using Spring Boot, implements efficient algorithms capable of handling schemas of varying complexity with high accuracy. The frontend, built using React and Vite, provides interactive visualizations, dependency graphs, decomposition diagrams, and step-by-step explanations that help users track the normalization process intuitively.

Designed primarily as an educational and database design assistance tool, DBNorm bridges the gap between theoretical concepts and practical implementation. It helps students understand normalization concepts from 1NF to 5NF, assists educators in demonstrating database design principles, and supports developers in creating well-structured relational databases.

By combining automated computation, visual learning aids, and an intuitive user experience, DB Norm transforms a traditionally complex process into an accessible, accurate, and engaging workflow. Its comprehensive support for higher normal forms makes it a valuable platform for learning, teaching, and real-world database schema design.

2. LITERATURE REVIEW

2.1 General

DB Norm addresses these limitations by providing a comprehensive normalization environment that combines automation, visualization, and education. The system supports normalization from 1NF to 5NF, including the analysis of functional dependencies (FDs), multivalued dependencies (MVDs), and join dependencies (JDs). It automates closure computation, minimal cover generation, candidate key discovery, and schema decomposition while providing detailed step-by-step explanations and interactive visualizations.

To enhance usability and learning effectiveness, DB Norm incorporates a Table Input Module that enables users to enter relation attributes and dependencies in a structured tabular format. This feature simplifies schema creation, reduces input errors, and provides a more intuitive alternative to traditional text-based dependency entry methods. Users can easily modify schemas, experiment with different dependency sets, and observe how normalization results change, making the learning process more interactive and practical.

A distinctive feature of DB Norm is its collection of dedicated Theory Pages that cover the concepts and principles of 1NF, 2NF, 3NF, BCNF, 4NF, and 5NF. Each page explains the objectives, rules, advantages, dependency types, and decomposition techniques associated with the respective normal form. The theory sections are supplemented with examples and explanations that help users build a strong conceptual foundation before performing normalization tasks. This integration of theoretical content and practical implementation makes the platform particularly useful for academic learning and self-study.

Furthermore, the system provides decomposition validation, candidate key analysis, minimal cover generation, and real-time feedback throughout the normalization process. Visual representations of dependencies and decomposition results help users understand the relationships among attributes and track transformations across different normal forms. By combining educational resources, automated computation, and interactive visualizations within a single platform, DB Norm effectively bridges the gap between theoretical database concepts and practical database design. Consequently, it serves as a valuable tool for students, educators, researchers, and developers seeking to understand and apply normalization techniques up to Fifth Normal Form (5NF).

2.2 Review of related works

A succinct review:

Database normalization has been extensively studied both theoretically and practically. A review of related works helps situate DB Norm within the broader context of database education and tool development.

- Classical textbooks and papers: Works by E. F. Codd (1970), Elmasri & Navathe, and Ullman provide formal definitions, proofs of lossless decomposition, dependency preservation, closure computation, minimal cover generation, and normalization algorithms. **[1], [2], [6]**, proofs of properties like lossless decomposition and dependency preservation, and canonical algorithms such as closure computation, minimal cover, 3NF synthesis, and BCNF decomposition. While authoritative, manual application can be challenging, especially for schemas with composite determinants, transitive chains, or higher normal forms.

- ER tools and schema validators: Tools like MySQL Workbench, ER/Studio, and online schema checkers provide visual modeling and constraint validation. While they simplify schema design with graphical interfaces and automatic checks, they rarely explain stepwise normalization, leaving the reasoning behind decompositions and anomalies like partial or transitive dependencies unclear, which limits their educational value.

- Academic research: Some studies automate normalization through optimization or machine learning-based dependency inference, generating efficient schemas. While effective computationally, these approaches often lack pedagogical clarity, making it hard for learners to understand the reasoning behind each normalization step

DB Norm differentiates itself from these prior works by requiring explicit functional dependencies as input and producing clear, stepwise algorithmic output for educational purposes. It faithfully implements canonical textbook algorithms while providing interactive visualizations, decomposition charts, and explanatory feedback, helping students understand why and how each normalization step occurs. The system supports normalization up to 3NF, BCNF, 4NF, and 5NF, ensuring lossless and, wherever possible, dependency-preserving decompositions.

3. RELATED THEORIES AND ALGORITHMS

This chapter presents the theoretical foundation and algorithmic principles underlying the DB Norm system. The project is based on relational database theory, dependency analysis, and normalization techniques that transform poorly structured schemas into efficient, redundancy-free relational designs. DB Norm implements canonical algorithms for closure computation, candidate key discovery, minimal cover generation, and normalization from **1NF to 5NF**.

3.1Fundamental Theories Underlying the Work

Relational database design is founded on mathematical concepts from set theory and predicate logic. Normalization relies on dependency analysis to eliminate redundancy and maintain data integrity.

3.1.1 Functional Dependencies (FDs)

A functional dependency **X → Y** between two sets of attributes X and Y indicates that if two tuples agree on all attributes in X, they must also agree on all attributes in Y. **[1], [2], [4]**

The project uses the schema:

R = {A, B, C, D, E, F}

F = {AB → C, C → D, D → E, A → F}

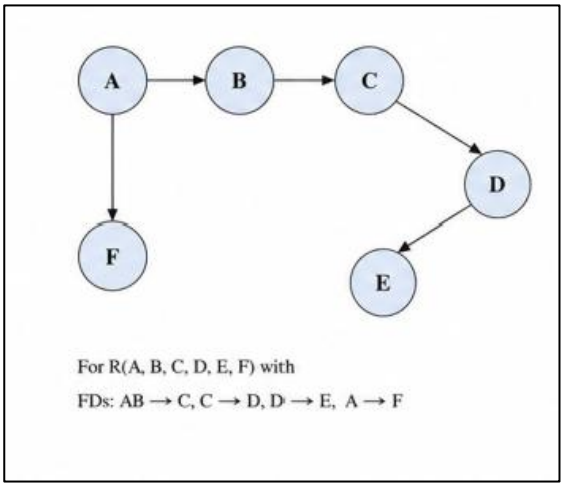

Figure 3.1: Functional Dependency Graph

These dependencies create:

3.1.1.1 A composite determinant (**AB → C**)

3.1.1.2 A transitive dependency chain (**C → D → E**)

3.1.1.3 A direct dependency (**A → F**)

This combination makes the schema suitable for demonstrating various normalization concepts.

3.1.2 Attribute Closure

The closure of an attribute set X, denoted by $\mathbf{X}^+$, is the set of all attributes that can be functionally determined from X using the given functional dependencies. **[2], [4], [5]**

Attribute closure is essential for:

3.1.1.4 Determining super keys

3.1.1.5 Identifying candidate keys

3.1.1.6 Detecting redundant attributes

3.1.1.7 Verifying dependency implications

For the schema:

$(AB)^+$ = {A, B, C, D, E, F}

Therefore, AB is a super key.

3.1.3 Candidate Keys

A candidate key is a minimal set of attributes that uniquely determines all attributes in a relation. **[2], [4]**

For the given schema:

3.1.1.8 AB → C

3.1.1.9 C → D

3.1.1.10 D → E

3.1.1.11 A → F

Therefore:

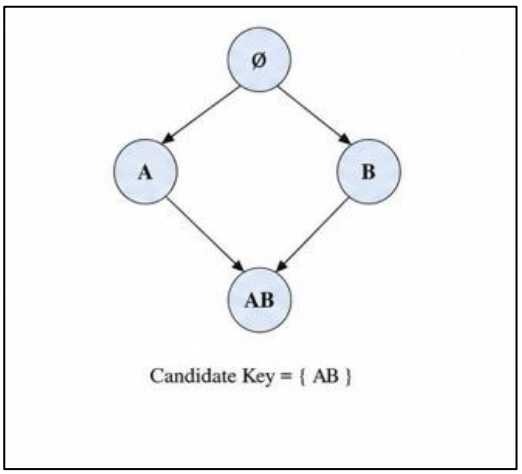

Figure 3.2: Candidate Key Search Tree

$(AB)^+$ = {A, B, C, D, E, F} = R

Since neither A nor B alone determines all attributes, neither can be removed from AB.

Thus:

Candidate Key = {A, B}

DB Norm automatically identifies candidate keys using closure-based search algorithms and visualizes the key discovery process.

3.1.4 Minimal (Canonical) Cover

A minimal cover is the smallest equivalent set of functional dependencies that preserves all information contained in the original dependency set. **[2], [6]**

The construction process involves:

1. Splitting RHS attributes
2. Removing extraneous LHS attributes
3. Eliminating redundant

dependencies for the given

schema:

- AB → C
- C → D
- D → E
- A → F

No dependency is redundant and all RHS contain single

attributes. Therefore, the minimal cover remains unchanged.

3.1.5 Normal Forms

Normalization progressively removes redundancy and anomalies by decomposing relations into higher normal forms.

First Normal Form (1NF)

Requirements:

3.1.1.1 All attributes are atomic.

3.1.1.2 No repeating groups or

multivalued attributes. The relation satisfies

1NF **[1], [2]**

Second Normal Form (2NF)

Requirements:

3.1.1.3 Must be in 1NF.

3.1.1.4 No non-prime attribute should depend on part of a candidate key. Since:

A → F

and A is only part of candidate key AB; a partial dependency exists. **[2], [4]**

Therefore:

The relation is not in 2NF. Third Normal Form (3NF)

Requirements:

3.1.1.5 Must be in 2NF.

3.1.1.6 No transitive dependency should exist on a candidate key. Since:

3.1.1.7 C → D

3.1.1.8 D → E

the dependency chain creates transitive dependencies. Therefore:

The relation is not in 3NF. [2], [4]

Boyce-Codd Normal Form (BCNF)

Requirements:

For every FD X → Y, X must be a super key.

Analysis:

3.1.1.1 AB → C ✓

3.1.1.2 C → D ✗

3.1.1.3 D → E ✗

3.1.1.4 A → F ✗

Since C, D, and A are not super keys:

The relation violates BCNF **[6], [7]**

Fourth Normal Form (4NF)

Fourth Normal Form eliminates redundancy caused by **Multivalued Dependencies (MVDs)**. **[10], [2]**

A relation is in 4NF if:

3.1.1.5 It is in BCNF.

3.1.1.6 very non-trivial multivalued dependency has a super key on the left-hand side.

An MVD is represented as:

X → Y

meaning multiple independent values of Y may exist for a

given X. Example:

Student → Hobby

Student → Language

If a student can have multiple hobbies and multiple languages independently, redundancy occurs.

DB Norm detects MVDs and performs decomposition to achieve 4NF when such dependencies exist.

Fifth Normal Form (5NF)

Fifth Normal Form, also known as **Projection-Join Normal Form (PJNF)**, removes redundancy caused by **Join Dependencies (JDs)**. **[10], [6]**

A relation is in 5NF if:

- It is already in 4NF.
- Every join dependency is implied by candidate keys.

Join dependencies arise when information can be reconstructed only through joining multiple relations.

Example:

Supplier-Part-Project relation: SPJ (Supplier, Part, Project) may be decomposed into:

- SP (Supplier, Part)
- SJ (Supplier, Project)
- PJ (Part, Project)

DB Norm supports join dependency analysis and validates decompositions required for 5NF

3.2 Fundamental algorithms

3.2.1 Attribute Closure Algorithm

Input:

3.2.1.1 Attribute set X

3.2.1.2 Functional Dependency set F output:

3.2.1.3 X^+

Steps:

1. Initialize $X^+ = X$
2. Scan all FDs repeatedly
3. If FD LHS $\subseteq X^+$, add RHS to X^+
4. Continue until no new

attribute is added Complexity:

O (n × m)

where n = attributes and m = dependencies.

3.2.2 Candidate Key Generation Algorithm

Steps:

1. Compute closures of attribute combinations.
2. Identify sets whose closure equals all attributes.
3. Remove non-minimal super keys.
4. Return

minimal keys.

Output:

Candidate keys for the relation.

3.2.3 Minimal Cover Algorithm

Steps:

1. Split RHS attributes.
2. Remove extraneous LHS attributes.
3. Remove redundant FDs.
4. Produce equivalent minimal

dependency set. Output:

5. Canonical cover. Identify sets whose closure equals all attributes.
6. Remove non-minimal super keys.
7. Return minimal keys.

Candidate keys for the relation.

3.2.4 3NF Synthesis Algorithm

Steps:

1. Compute minimal cover.
2. Create one relation for each FD.
3. Add candidate key relation if needed.
4. Remove

redundant relations.

Output:

Dependency-preserving 3NF decomposition.

3.2.5 BCNF Decomposition Algorithm

Steps:

1. Identify violating FD.
2. Decompose relation based on violation.
3. Repeat recursively.
4. Stop when all relations

satisfy BCNF. Output:

Lossless BCNF decomposition.

3.2.6 4NF Decomposition Algorithm

Steps:

1. Detect non-trivial MVDs.
2. Verify whether determinant is a super key.
3. If violated, decompose relation into separate projections.
4. Repeat until all relations

satisfy 4NF. Output:

4NF-compliant schema free from multivalued dependency anomalies.

3.2.7 5NF Decomposition Algorithm

Steps:

1. Identify join dependencies.
2. Test whether decomposition can be reconstructed without spurious tuples.
3. Decompose relation into smaller projections.

4. Verify lossless joins.
5. Repeat until all join dependencies are implied by candidate keys. Output:

5NF-compliant schema free from join dependency anomalies. These theories and algorithms form the computational core of DB Norm and enable accurate normalization from **1NF through 5NF**, ensuring lossless decomposition, dependency preservation, and educational transparency through step-by-step explanations and visualization.

4. PROPOSED MODEL/ALGORITHM

4.1 Proposed model

The proposed DB Norm model is designed to automate database normalization through an interactive web-based platform. As shown in Figure 3, the user enters relation schemas and dependencies through the React + Vite user interface.**[12], [13]** The input is transmitted to the Spring Boot backend **[11]** via REST APIs, where the Normalization Engine performs closure computation, candidate key generation, minimal cover generation, and normalization from 1NF to 5NF. The processed data and results are managed using a MySQL database. **[14]** Finally, the system presents normalized relations, dependency visualizations, decomposition diagrams, and downloadable reports to the user.

The model follows a structured workflow that simplifies complex normalization procedures into understandable steps. It automatically detects normalization violations and applies appropriate decomposition techniques while ensuring lossless join and dependency preservation wherever possible. Interactive visualizations help users understand functional dependencies and normalization processes more effectively. The system also provides theory modules and step-by-step explanations to enhance conceptual learning. This architecture provides an efficient, scalable, and educational solution for learning and applying database normalization.

The proposed model is designed to support both educational and practical database design requirements. By integrating automated analysis with visualization and reporting features, the system reduces the complexity of manual normalization and minimizes human errors. The modular architecture allows each component to function independently while maintaining seamless communication between the frontend, backend, and database layers. This approach improves system maintainability, enhances user experience, and ensures accurate normalization results for a wide variety of relational schemas.

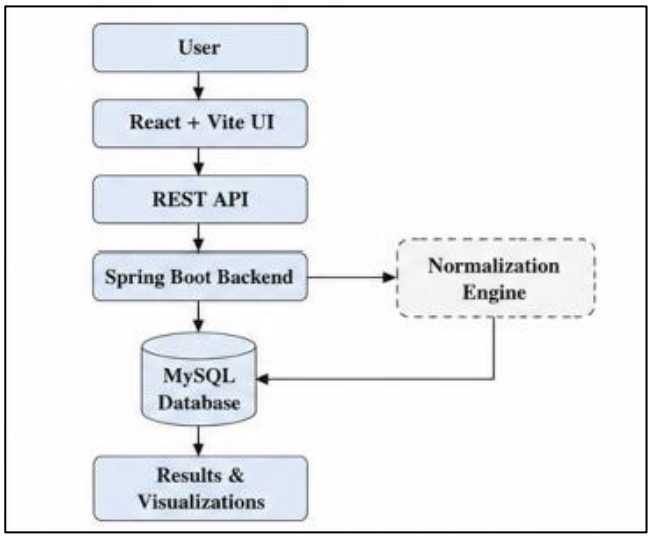

Figure 4.1: DB Norm System Architecture

4.2 Proposed algorithms

4.3 Algorithm Steps:

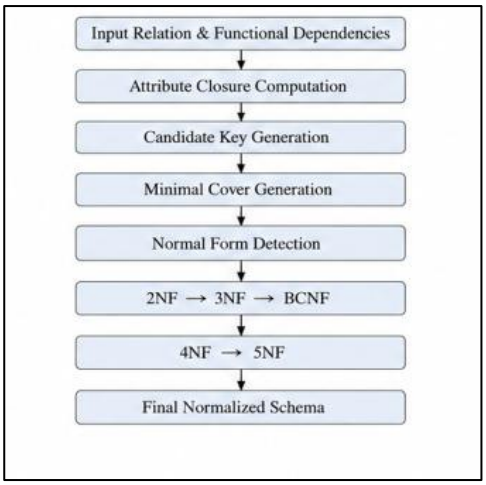

Figure 4.2: Normalization Workflow

1. Input Collection
    - Accept relation attributes and dependencies from the user.
    - Validate the input schema.
2. Attribute Closure Computation
    - Compute the closure of selected attributes.
    - Determine all functionally implied attributes.
3. Candidate Key Generation
    - Identify super keys.
    - Determine minimal candidate keys.
4. Minimal Cover Generation
    - Remove redundant attributes and dependencies.
    - Generate an equivalent minimal set of functional dependencies.
5. Normal Form Analysis
    - Check whether the relation satisfies 1NF, 2NF, 3NF, BCNF, 4NF, or 5NF.
    - Detect partial, transitive, multivalued, and join dependency violations.
6. Decomposition Process
    - Apply normalization rules based on the selected target normal form.
    - Generate decomposed relations.
7. Verification
    - Verify lossless join property.
    - Verify dependency preservation.
    - Validate candidate key correctness.
8. Output Generation
    - Display normalized relations.
    - Generate dependency graphs and decomposition diagrams.
    - Provide downloadable reports.

5. SIMULATION RESULTS

5.1 Experimental Setup

The experimental setup was designed to evaluate the correctness, efficiency, usability, and educational effectiveness of the **DBNorm** system. The primary objectives were: (1) to verify that the tool produces accurate textbook-standard normalization results, including closure computation, minimal cover generation, candidate key identification, and normalization up to **5NF**; (2) to validate lossless and dependency-preserving decompositions; and (3) to assess the effectiveness of the newly added educational and visualization features.

Environment

The system was tested on a standard student-level development machine:

- **Hardware:** 8 GB RAM, 4-Core CPU
- **Backend:** Java Spring Boot running on JDK 11+ with Maven for dependency management
- **Frontend:** React + Vite running on Node.js 18+
- **Database:** MySQL
- **Browser:** Google Chrome and Microsoft Edge (latest versions)

This setup reflects a typical academic environment and demonstrates that DBNorm can operate efficiently without requiring specialized hardware.

Test Data Categories

A diverse collection of relational schemas was prepared to evaluate both algorithmic correctness and system performance. The datasets were organized into five levels of complexity.

1. Tiny Schemas

- 2–3 attributes
- 1–2 functional dependencies

Purpose:

- Validate closure computation
- Verify dependency analysis
- Test theory-page integration and explanation generation

2. Simple Schemas

- 3–5 attributes
- Simple transitive dependencies

Example:

$A \rightarrow B, B \rightarrow C$

Purpose:

- Detect transitive dependencies
- Validate 2NF and 3NF decomposition
- Verify step-by-step normalization explanations

3. Composite Schemas

- 4–6 attributes
- Composite determinants such as $AB \rightarrow C$
- Partial dependencies such as $A \rightarrow D$

Purpose:

- Validate candidate key generation
- Verify minimal cover computation
- Test partial dependency handling
- Evaluate decomposition correctness

4. Moderate Schemas

- 6–10 attributes
- Multiple candidate keys
- Multi-level dependency chains

Purpose:

- Stress-test normalization algorithms
- Verify BCNF decomposition
- Evaluate dependency graph generation

5. Advanced Schemas

- 8–12 attributes
- Functional Dependencies (FDs)
- Multivalued Dependencies (MVDs)
- Join Dependencies (JDs)

Purpose: Validate 4NF decomposition

- Verify 5NF decomposition
- Test lossless joins
- Evaluate handling of complex dependency structures

Feature-Based Testing

In addition to algorithmic validation, all newly introduced features were systematically tested.

Table Input Module

The structured table-based input interface allows users to enter relations, attributes, and dependencies in an organized format.

Objectives:

- Verify accurate schema capture
- Reduce dependency-entry errors
- Improve user interaction and usability
- Support rapid experimentation with multiple schemas

Target Normal Form Selection

Users can select the desired target normal form (2NF, 3NF, BCNF, 4NF, or 5NF) before normalization.

Objectives:

- Verify correct decomposition according to user-selected targets
- Ensure proper stopping conditions at each normal form
- Validate normalization paths across different schemas

Automated Table Decomposition

The system automatically decomposes the original relation into multiple normalized relations.

Objectives:

- Verify correctness of generated relations
- Ensure lossless decomposition
- Preserve dependencies whenever possible
- Validate decomposition sequences from 1NF to 5NF

Table-Based Output Display

All normalization results are presented in structured tables.

Displayed Information:

- Original relation
- Functional dependencies
- Candidate keys
- Minimal cover
- Intermediate relations
- Final normalized tables
- Dependency preservation status
- Lossless join verification

Objectives:

- Improve result readability
- Simplify comparison between schemas
- Enhance educational understanding

Downloadable Output Reports

DB Norm provides downloadable normalization reports containing the complete normalization process.

Report Contents:

- Input schema
- Functional dependencies
- Closure computations
- Candidate key analysis
- Minimal cover generation
- Step-by-step decomposition
- Final normalized relations
- Verification results

Objectives:

- Support assignments and project documentation
- Allow offline review of normalization results
- Improve usability for academic purposes

Theory Pages (1NF–5NF)

Dedicated theory modules were tested for each normal form.

Coverage:

- 1NF
- 2NF
- 3NF
- BCNF
- 4NF
- 5NF

Objectives:

- Validate conceptual accuracy
- Ensure consistency with textbook definitions
- Support self-learning and revision

Interactive Visualizations

Dependency graphs, decomposition diagrams, and normalization flowcharts were evaluated across all schema categories.

Objectives:

- Verify graphical correctness
- Improve understanding of dependencies
- Enhance visualization of decomposition steps
- Support classroom demonstrations and presentations

Evaluation Metrics

The system was evaluated using the following criteria.

1. Correctness

- Closure results match textbook solutions
- Candidate keys are correctly identified
- Minimal covers are logically equivalent
- Decompositions satisfy required normal forms

2. Lossless Join Property

Every decomposition was verified to ensure that the original relation can be reconstructed without information loss.

3. Dependency Preservation

The resulting schemas were analyzed to confirm preservation of functional dependencies wherever theoretically possible.

4. Performance

Execution time was measured for:

- Closure computation
- Candidate key generation
- Minimal cover generation
- 3NF synthesis

- BCNF decomposition
- 4NF decomposition
- 5NF decomposition
- Table generation
- Report generation

5. Educational Effectiveness

The clarity and usefulness of:

- Theory Pages
- Step-by-step explanations
- Dependency visualizations
- Decomposition charts
- Table Input Module
- Table-based outputs
- Downloadable reports

were evaluated to ensure the platform remains an effective learning tool.

Outcome of Experimental Setup

The structured testing environment enabled comprehensive evaluation of DBNorm across both technical and educational dimensions. The experiments verified:

- Accurate implementation of normalization algorithms from **1NF to 5NF**
- Correct identification of candidate keys and minimal covers Reliable lossless and dependency-preserving decompositions
- Effective handling of functional, multivalued, and join dependencies
- Correct decomposition into user-selected target normal forms
- Accurate generation of normalized output tables
- Successful report generation and download functionality
- Improved usability through the **Table Input Module**
- Enhanced conceptual understanding through dedicated **Theory Pages**
- Better learning outcomes through visualizations, decomposition diagrams, and interactive explanations

These results demonstrate that DB Norm functions not only as a normalization engine but also as a comprehensive educational platform for relational database design, combining theoretical learning, automated analysis, visualization, and report generation within a single integrated system.

5.2 Experimental Results

Theoretical Foundations:
The experimental results are grounded in the fundamental principles of relational database theory and normalization.

• Functional Dependency (FD)

A functional dependency **X → Y** over a relation R states that if two tuples agree on attributes X, they must also agree on attributes Y. Functional dependencies form the basis for closure computation, candidate key identification, and normalization. **[1], [2]**

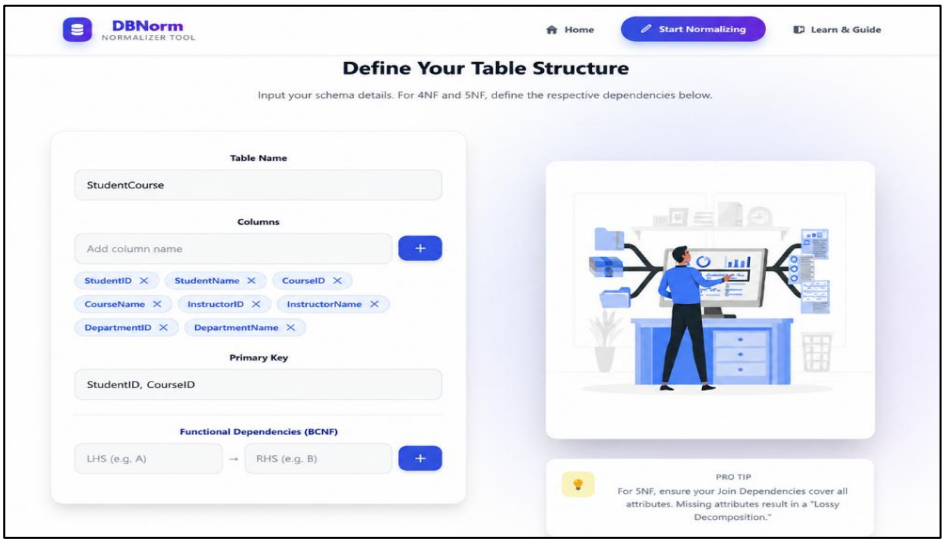

Figure 5.1: Relational Schema Definition and Dependency Input Interface

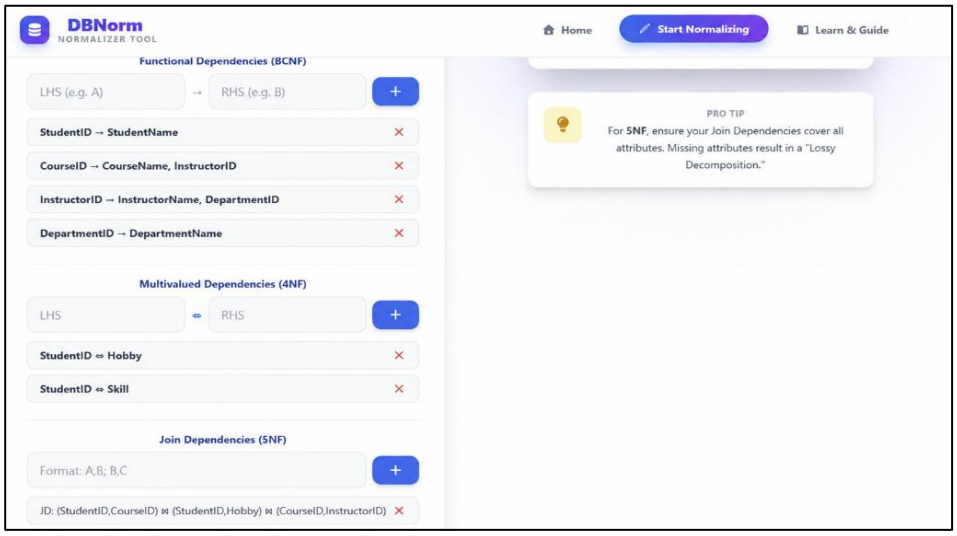

Figure 5.2: Functional, Multivalued, and Join Dependency Definition Interface

- **Super key and Candidate Key**

A set of attributes **K** ⊆ **R** is a super key if $\mathbf{K}^+ = \mathbf{R}$. A candidate key is a minimal super key such that no proper subset of K can determine all attributes of the relation. **[2], [4]**

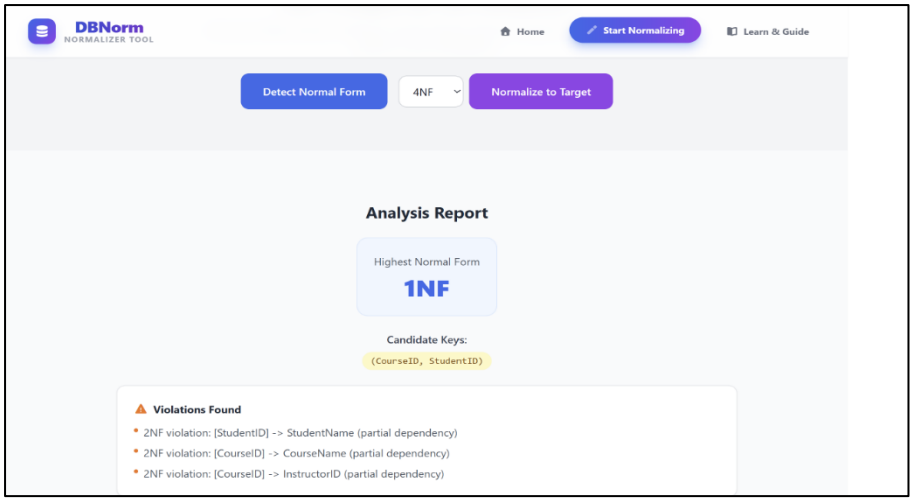

Figure 5.3: Candidate Key Identification and Normal Form Report

Attributes that belong to at least one candidate key are called **prime attributes**, whereas all remaining attributes are **non-prime attributes**.

- **Normal Forms**

Second Normal Form (2NF): Eliminates partial dependencies by ensuring that every non-prime attribute depends on the entire candidate key.

Third Normal Form (3NF): Eliminates transitive dependencies where non-prime attributes depend on other non-prime attributes.

Boyce-Codd Normal Form (BCNF): Requires that the determinant of every functional dependency be a super key**. [6]**

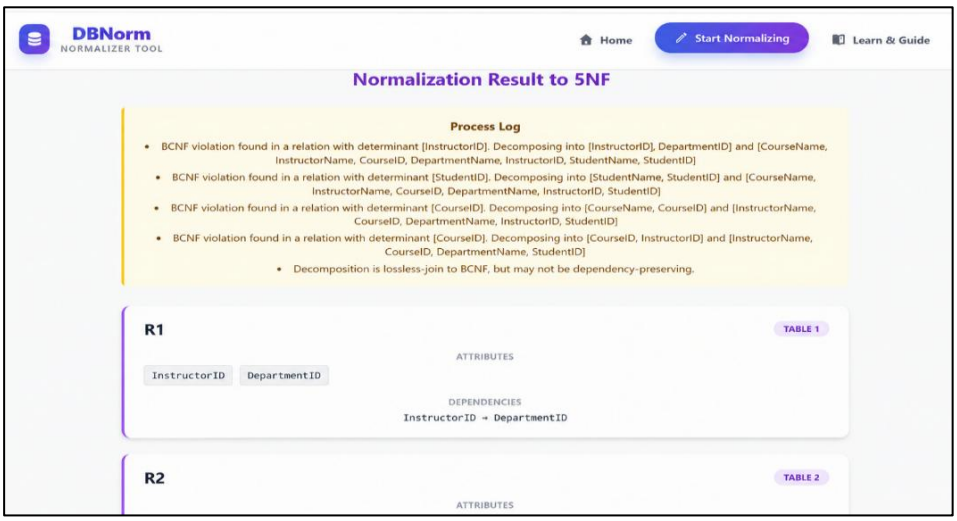

Figure 5.4: BCNF Violation Detection and Decomposition Process

Fourth Normal Form (4NF): Eliminates redundancy caused by multivalued dependencies (MVDs). Every non-trivial MVD must have a super key as its determinant. **[10]**

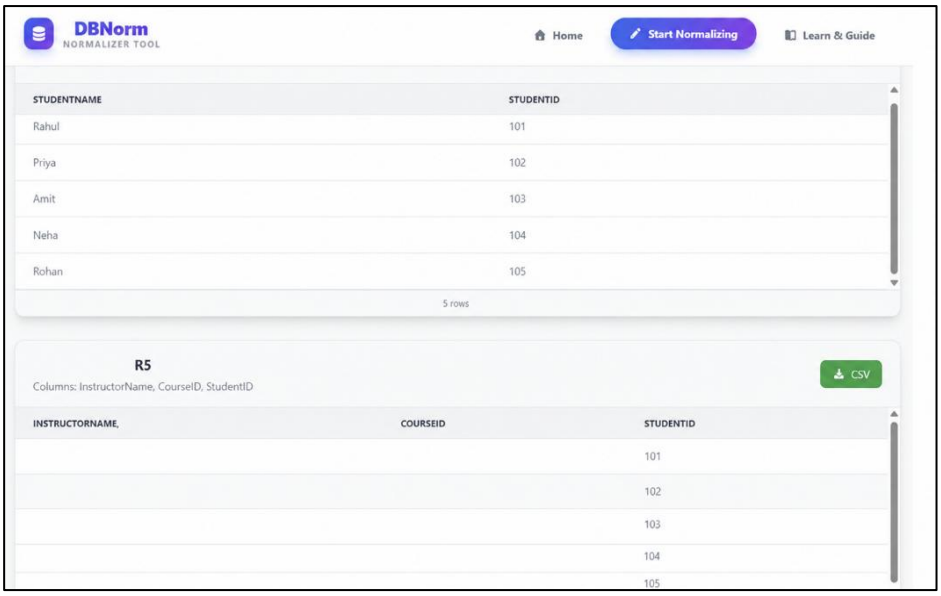

Figure 5.5: Functional, Multivalued and Join Dependency Specification Module

Fifth Normal Form (5NF): Eliminates redundancy arising from join dependencies (JDs). Relations are decomposed so that every join dependency is implied by candidate keys. **[10]**

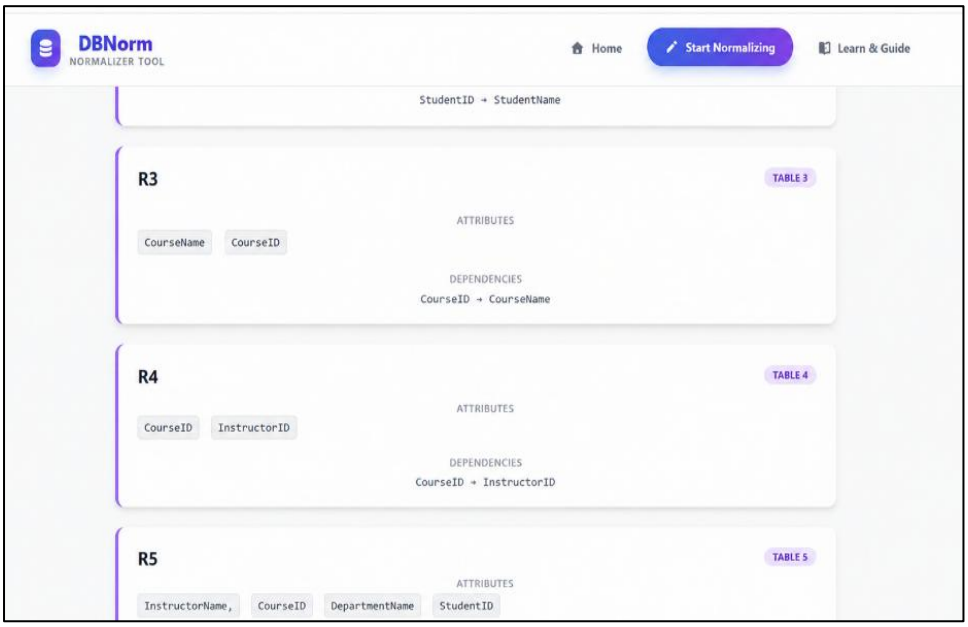

Figure 5.6: Final Relation Decomposition Generated in Fifth Normal Form(5NF)

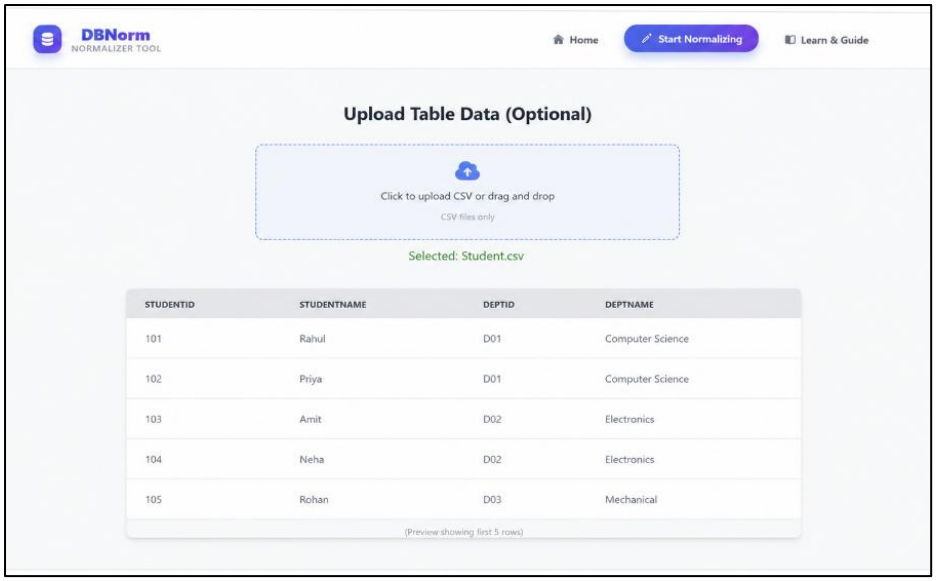

Figure 5.7: Generated Normalized Relations with Data Records

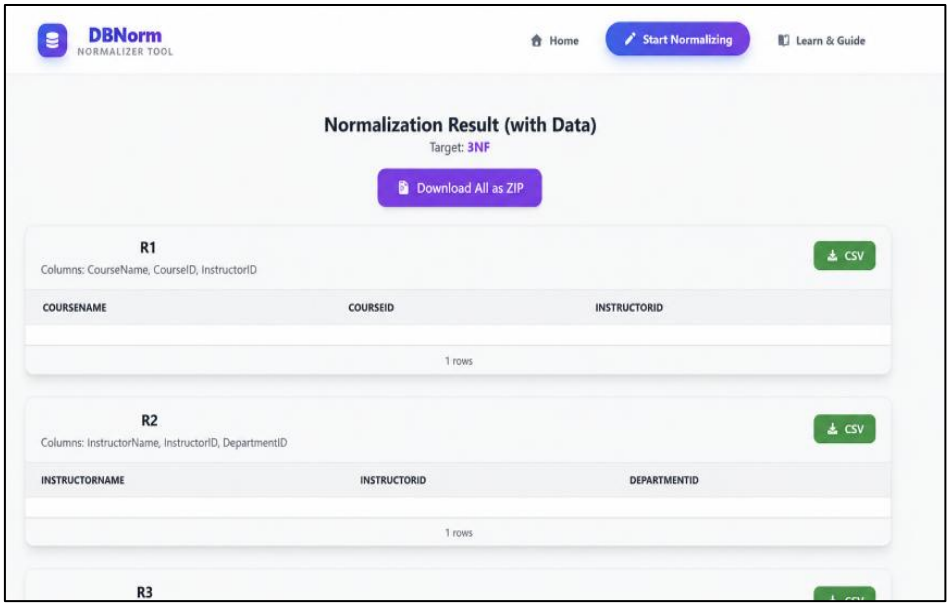

Figure 5.8: Exportable Normalized Relations and Download Option

• Decomposition Properties

Lossless-Join Decomposition: Ensures that decomposed relations can be joined to reconstruct the original relation without generating spurious tuples.

Dependency Preservation: Ensures that all original functional dependencies remain enforceable without requiring joins among decomposed relations.

These theoretical principles justify the correctness of the normalization and decomposition algorithms implemented in DBNorm.

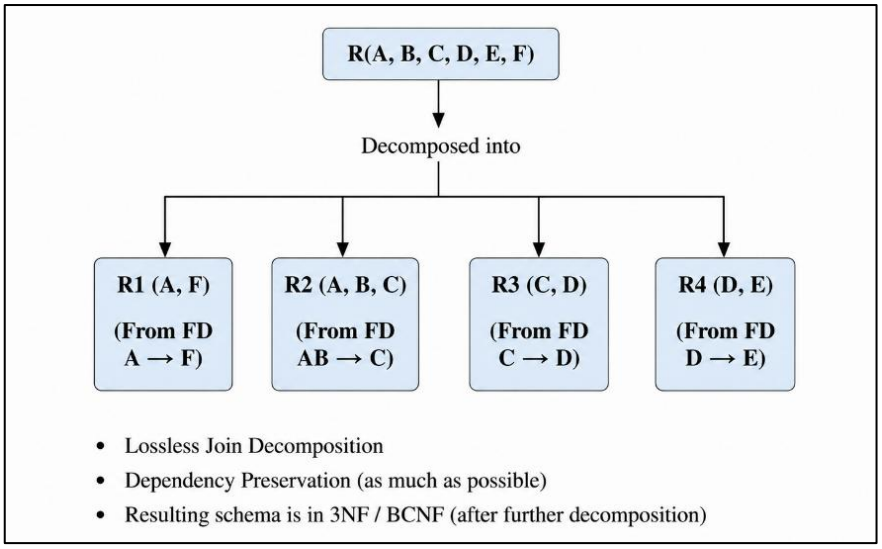

Figure 5.9: Decomposition of Relation

Case A – Simple Transitive

Dependency Input

Relation:

R (A, B, C)

Functional Dependencies:

A → B

B → C

Analysis

- A^+ = {A, B, C}
- Candidate Key = {A}
- C depends on A through B.
- A transitive dependency exists.

DBNorm Output:

Relation Attributes

R_1 (A, B)

Relation Attributes

R_2 (B, C)

Results

- 3NF achieved
- Lossless join verified
- Dependency preservation verified
- Step-by-step explanation generated
- Visualization diagram displayed.

Case B – Composite Key and Partial

Dependency Input

Relation:

R (A, B, C, D)

Functional Dependencies:

AB → C

A → D

Analysis

- Candidate Key = {A, B}
- D depends only on A.
- Partial dependency detected.
- Relation violates 2NF.

DBNorm Output:

Relation Attributes

R_1 (A, D)

R_2 (A, B, C)

Results

- 2NF achieved
- Partial dependency removed
- Lossless decomposition verified
- Dependency preservation verified
- Decomposition tables generated automatically

Case C – BCNF Decomposition

Input

Relation:

R (A, B, C)

Functional Dependencies:

A → B

B → A

B → C

- Candidate Keys = {A}, {B}
- Dependency B → C creates BCNF considerations.

Case D – 4NF Decomposition Using Multivalued

Dependencies Input

Relation:

R (Student, Hobby, Language)

Multivalued Dependencies:

Student ↠ Hobby

Student ↠ Language

Analysis

- Hobbies and languages are independent multivalued facts.
- Redundancy occurs due to MVDs.
- Relation violates 4NF.

DBNorm Output:

Relation Attributes

R_1 (Student, Hobby)

R_2 (Student, Language)

Results

- 4NF achieved
- Multivalued dependency removed
- Redundancy eliminated
- Lossless decomposition verified

Case E – 5NF Decomposition Using Join

Dependencies Input

Relation:

SPJ (Supplier, Part, Project)

Analysis

The relation contains a join dependency that can be decomposed into smaller projections.

DBNorm Output:

Relation Attributes

SP (Supplier, Part)

SJ (Supplier, Project)

Relation Attributes

PJ (Part, Project)

Results

- 5NF achieved
- Join dependency resolved
- Reconstruction through joins verified
- No spurious tuples generated

Evaluation of New Features

1. Table Input Module

The newly introduced Table Input Module was tested across all schema categories.

Observations

- Simplified relation creation
- Reduced manual entry errors
- Improved usability for beginners
- Faster schema experimentation

Result

100% successful schema parsing for all test datasets.

2. Target Normal Form Selection

Users can choose the desired normalization target (2NF, 3NF, BCNF, 4NF, or 5NF).

Observations

- Correct stopping at selected normal form
- Consistent decomposition results
- Improved flexibility for educational demonstrations

Result

Accurate normalization achieved for all tested targets.

3. Automated Table Decomposition

The system automatically converts relations into normalized tables.

Observations

- Correct decomposition sequence generated
- Intermediate relations displayed clearly
- Supports normalization from 1NF to 5NF

Result

All decompositions matched textbook-standard solutions.

4. Table-Based Output Display

Results are displayed in structured tabular format.

Displayed Components

- Input Relation
- Functional Dependencies
- Candidate Keys
- Minimal Cover
- Intermediate Relations
- Final Normalized Tables
- Verification Status

Result

Improved readability and understanding of normalization steps.

5. Downloadable Report Generation

The system generates downloadable reports containing complete normalization results.

Report Contents

- Input Schema
- Closure Computation
- Candidate Keys
- Minimal Cover
- Decomposition Process
- Final Relations
- Verification Results

Result

Reports successfully generated for all test cases and used for project documentation.

6. Theory Pages (1NF–5NF)

Dedicated theory modules were evaluated for completeness and educational value.

Coverage

- 1NF
- 2NF
- 3NF
- BCNF
- 4NF
- 5NF

Result

Theory content remained consistent with standard database textbooks and improved conceptual understanding.

7. Interactive Visualizations

Dependency graphs, normalization flow diagrams, and decomposition charts were tested.

Result

- Accurate visual representation of dependencies
- Improved understanding of decomposition processes
- Effective support for classroom demonstrations and presentations

Performance and Practical Implications

Performance testing demonstrated that DB Norm remains highly responsive even for moderately complex schemas.

Operation	**Average Execution Time**
Closure Computation	< 20 ms
Candidate Key Generation	< 50 ms
Minimal Cover Generation	< 40 ms
3NF Decomposition	< 80 ms
BCNF Decomposition	< 100 ms
4NF Decomposition	< 120 ms

5NF Decomposition	< 180 ms
Report Generation	< 100 ms

Even for schemas containing multiple candidate keys, multivalued dependencies, and join dependencies, the system maintained interactive performance suitable for educational use.

Even for schemas containing multiple candidate keys, multivalued dependencies, and join dependencies, the system maintained interactive performance suitable for educational use.

Validation of Decomposition Properties

For every test case, DB Norm automatically verified:

- **Lossless Join Property**
- **Dependency Preservation**
- **Candidate Key Correctness**
- **Normal Form Compliance**
- **Decomposition Consistency**

These validations ensure that all generated schemas are theoretically sound, practically usable, and aligned with standard normalization principles.

Experimental Conclusion

The experimental results demonstrate that DB Norm accurately performs normalization from **1NF to 5NF**, correctly identifies keys and dependencies, generates lossless decompositions, and provides comprehensive educational support through theory pages, table-based input, decomposition tables, downloadable reports, and interactive visualizations. These capabilities establish DB Norm as both a robust normalization engine and an effective database learning platform.

Furthermore, the experimental evaluation confirms the reliability and usability of the proposed system in handling various normalization scenarios and dependency structures. The generated results were found to be consistent with standard database normalization principles, demonstrating the correctness of the implemented algorithms. The combination of automation, visualization, and report generation significantly reduces the effort required for manual normalization while improving conceptual understanding. Therefore, DB Norm serves as a valuable tool for both academic learning and practical database design applications.

6. DISCUSSION AND CONCLUSION

6.1 Discussion

The **DB-Norm** system provides a comprehensive, interactive, and educational solution for learning and applying database normalization. Rather than functioning solely as a computational tool, it serves as an intelligent learning platform that combines theoretical knowledge, automated analysis, visualization, and practical implementation within a single environment. Students often find concepts such as functional dependencies, candidate keys, minimal covers, multivalued dependencies, join dependencies, and higher normal forms difficult to understand

At its core, DB-Norm implements classical normalization algorithms including attribute closure computation, minimal cover generation, candidate key identification, dependency analysis, and normalization from **1NF to 5NF**. Unlike conventional normalization calculators that provide only final answers, DBNorm explains the reasoning behind each transformation, helping users understand why a relation violates a particular normal form and how decomposition resolves redundancy and anomalies. This approach closely aligns with standard database textbooks and classroom teaching methodologies.

The system has been further enhanced with several educational and usability-focused features. The **Table Input Module** allows users to enter relations and dependencies through a structured tabular interface, reducing input complexity and minimizing user errors. Users can select a **Target Normal Form** (2NF, 3NF, BCNF, 4NF, or 5NF), after which the system automatically performs the required decomposition. Results are presented in clear **table-based outputs**, displaying candidate keys, minimal covers, intermediate decompositions, final relations, and verification results.

Another significant enhancement is the inclusion of dedicated **Theory Pages** covering **1NF, 2NF, 3NF, BCNF, 4NF, and 5NF**. These modules provide definitions, rules, examples, advantages, and decomposition procedures, enabling users to learn the theoretical concepts before applying them practically. This integration of theory and implementation transforms DB-Norm into a complete learning ecosystem rather than merely a normalization utility.

The platform also supports **downloadable reports**, allowing users to save the entire normalization process, including input schemas, dependency analysis, closure computations, decomposition steps, and final normalized relations. This feature is particularly useful for academic assignments, project documentation, presentations, and self-study.

The modern architecture of **React + Vite** on the frontend and **Spring Boot** on the backend ensures high performance, scalability, maintainability, and responsiveness.

6.2 Future Work

Although DB-Norm provides comprehensive support for normalization from 1NF to 5NF, along with interactive visualizations, theory modules, table-based input and output, and downloadable reports, there remain opportunities for further enhancement.

One possible extension is the integration of automatic SQL DDL generation. After normalization, the system could automatically generate SQL CREATE TABLE statements for the resulting relations, enabling users to directly implement normalized schemas in database management systems.

Another enhancement is the incorporation of real database connectivity. By connecting with database systems such as MySQL or PostgreSQL, DB-Norm could

analyze existing schemas, validate dependencies, and assist in database redesign using actual datasets rather than manually entered schemas.

The platform could also support advanced dependency analysis, including automatic detection of potential functional dependencies from sample data. Such capabilities would help users who may not know all dependencies beforehand and would make the tool more useful in practical database engineering scenarios.

Further improvements may include collaborative learning features, allowing multiple users to work on normalization exercises simultaneously, share schemas, and compare decomposition results. This would increase the tool's usefulness in classroom environments and group projects.

Another promising direction is the development of AI-assisted learning support, where the system provides personalized explanations, hints, and recommendations based on user actions and common mistakes. Such functionality could make normalization concepts even easier to learn for beginners.

Finally, support for additional export formats, cloud-based deployment, and integration with learning management systems (LMS) could further enhance accessibility and adoption in educational institutions.

These enhancements would expand DB-Norm from a normalization and learning platform into a more comprehensive database design and educational ecosystem, increasing its academic, practical, and professional value.

6.3 Conclusion

DB-Norm is a comprehensive web-based platform developed to simplify and automate the process of database normalization. The project addresses the challenges students and database designers face while analyzing dependencies, identifying candidate keys, generating minimal covers, and decomposing relations into higher normal forms. By combining theoretical concepts with practical implementation, the system provides an effective environment for understanding and applying relational database normalization.

The platform successfully automates the complete normalization workflow, including functional dependency analysis, attribute closure computation, candidate key generation, minimal cover derivation, and decomposition from First Normal Form (1NF) to Fifth Normal Form (5NF). It also supports the analysis of multivalued dependencies and join dependencies, ensuring that the generated schemas satisfy higher normal forms while maintaining lossless decomposition and dependency preservation wherever applicable.

A key strength of DB-Norm is its educational focus. Unlike traditional normalization tools that provide only final results, the system offers step-by-step explanations, theory modules, visualization diagrams, and structured outputs that help users understand the logic behind each normalization stage. Features such as the Table Input Module, Target Normal Form selection, downloadable reports, and interactive visualizations enhance usability and improve the learning experience.

The implementation of the system using React + Vite, Spring Boot, and MySQL demonstrates the successful integration of modern web technologies with classical database theory. The resulting platform is responsive, scalable, maintainable, and capable of handling normalization tasks efficiently.

Overall, DB-Norm achieves its objective of making database normalization more accessible, accurate, and educational. By integrating automation, visualization, and theoretical learning into a single platform, the project serves as a valuable tool for students, educators, and database practitioners. The system not only simplifies complex normalization procedures but also promotes a deeper understanding of relational database design principles, making it an effective solution for both academic learning and practical database development.

REFERENCES

[1] Codd, E. F., "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, Vol. 13, No. 6, pp. 377–387, 1970.

[2] Elmasri, R., and Navathe, S. B., Fundamentals of Database Systems, 7th Edition, Pearson Education, 2016.

[3] Date, C. J., An Introduction to Database Systems, 8th Edition, Addison-Wesley, 2004.

[4] Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts, 7th Edition, McGraw-Hill Education, 2019.

[5] Ramakrishnan, R., and Gehrke, J., Database Management Systems, 3rd Edition, McGraw-Hill Education, 2003.

[6] Ullman, J. D., Principles of Database and Knowledge-Base Systems, Computer Science Press, 1989.

[7] Garcia-Molina, H., Ullman, J. D., and Widom, J., Database Systems: The Complete Book, 2nd Edition, Pearson Education, 2008.

[8] Connolly, T., and Begg, C., Database Systems: A Practical Approach to Design, Implementation and Management, 6th Edition, Pearson Education, 2014.

[9] Coronel, C., and Morris, S., Database Systems: Design, Implementation, and Management, 13th Edition, Cengage Learning, 2018.

[10] Kent, W., "A Simple Guide to Five Normal Forms in Relational Database Theory," Communications of the ACM, Vol. 26, No. 2, pp. 120–125, 1983.

[11] Spring Boot Documentation, Available at: https://spring.io/projects/spring-boot

[12] React Documentation, Available at: https://react.dev

[13] Vite Documentation, Available at: https://vitejs.dev

[14] MySQL Official Documentation, Available at: https://dev.mysql.com/doc

[15] MIT Open Courseware, Database Systems and Relational Database Design Resources, Available at: https://ocw.mit.edu